KSP-H 38-2-3 Opisování

Řešení:

Jelikož jediným cílem, kterého chce Petr dosáhnout je co **nejnižší** počet studentů kteří zkoušku splní (mají plný počet bodů), stačí nalézt řešení, které doručí právě tento požadovaný výsledek. Proto nám klidně postačí, aby všichni studenti nevěděli tu stejnou a jedinou otázku a neprojde nikdo. Nemusíme tak díky Petrovým přísným kritériím brát v úvahu kolik dalších otázek student v testu nevyplnil.

Algoritmus proto zjistí ke každé jednotlivé otázce v testu, kolik studentů ji neví. Následně podle otázky, kterou ví nejméně studentů (její pozici v tabulce na vstupu uchováme v proměnné ***index***), posadí všechny studenty, kteří u ní odpověď nevědí dopředu před ty, kteří odpověď vědí nehledě na pořadí mezi nimi (stačí pouze dodržet, aby všichni, co tu otázku nevědí byly před těmi, co ji vědí). To je celé. Jen dodám, že v případě kdy je více otázek, které mají stejný největší počet studentů, kteří je nevědí, algoritmus vybere libovolnou z nich (konkrétně tu první), podle které studenty usadí. To samé platí, pokud by všichni studenti věděli všechny otázky.

Důkaz:

Abych dokázal, že tento postup vždy zaručí co nejmenší počet úspěšných řešitelů u zkoušky, definuji už správně rozmístěnou řadu ***R*** (***R[0]*** bude první student v řadě a ***R[N]*** poslední), a pro ni doložím následující tvrzení:

1) Pro neúspěch u zkoušky stačí jediná nezodpovězená otázka
2) Jakmile se na určitém místě ***R[i]*** v řadě nachází student, který zná odpověď na otázku ***M[j]***, budou ji znát všichni studenti sedící za ním až po ***R[N]***, jelikož ji opíšou

Z toho vyplývá, že rozmísťovat studenty podle otázek, na které už někdo před nimi zná odpověď, nemá cenu. Otázku ***M[j]*** nemohou opsat studenti, jejichž spolužáci sedící před nimi ji také nemohou opsat a správnou odpověď neznají. To nastane v případě, kdy před nimi nikdo odpověď na otázku ***M[j]*** nezná. Proto se v optimálním rozmístění otázka ***M[j]*** stává právě tou (***M[index]***) podle které studenty rozmísťovat, pokud ze všech ***M*** otázek ji v optimálním rozmístění může opsat co nejméně studentů.

Detailnější popis algoritmu:

Algoritmus získá na vstupu tabulku ***ST*** reprezentovanou v 2D poli, kde velikost prvního rozměru bude počet studentů ***N*** a druhý rozměr bude velikosti počtu otázek ***M***. Dále si vytvoříme pomocná pole algoritmu ***D_ST*** (bude uchovávat pozice studentů z ***ST***, kteří odpověď na otázku ***M[index]*** nevědí), ***S_ST*** (uchovává pozice studentů z ***ST***, kteří odpověď na otázku ***M[index]*** vědí) a ***QUEST*** (pole o velikosti ***M*** uchovávající na pozici ***M[j]*** počet všech studentů, kteří odpověď na tuto otázku neznají).

Poté jednoduše projdu vstup jedním vnořeným cyklem a pokaždé zkontroluji zda platí ***ST[i][j] == 0***. Pokud ano, přičtu v poli prvek ***QUEST[J]*** o jeden. Až projdu celý vstup a v průběhu aktualizuji správně ***QUEST***, projdu pole ***QUEST*** znovu a získám ***index***, na kterém je v poli největší hodnota.

Poté projdu vstup respektive studenty ještě jednou, tentokrát ovšem budu pouze zjišťovat, zda daný student odpověď na otázku na pozici ***index***, zná nebo ne. Pokud bude platit ***ST[i][index] == 0***, tak jako hodnotu vložím do ***D_ST*** pozici žáka v tabulce, který odpověď nezná. V opačném případě pozici žáka v tabulce vložím do ***S_ST***.

Pak už stačí pouze obě pomocná pole ***D_ST*** a ***S_ST*** spojit do prvního tak, že první v řadě budou studenti z ***D_ST*** a až poté studenti z ***S_ST*** a takto spojené pole vrátit jako finální výsledek.

Typ výstupu bude pole jehož číslování prvků reprezentuje finální řadu (prvek na pozici 0 je první student v optimální řadě) a hodnota na pozici ***i*** říká, který student ze vstupu se má zde usadit.

Př. výpisu optimálního rozmístění →
Pro i = 0 procházej dokud platí i < N, po každém opakování přičti 1 do i:
Vypiš(ST[D_ST[i]])

Asymptotická časová a prostorová složitost:

Časová složitost bude ***O(NM)*** jelikož musíme u každého studenta zjistit, na které otázky odpvěď zná a na které ne. V ostatních případech procházíme zvlášť buď studenty, u kterých zjišťujeme zda na optimální otázku odpověď znají → ***O(N)*** nebo otázky u kterých hledáme tu, jejíž odpověď nezná nejvíce studentů → ***O(M)***.

Prostorová složitost bude taktéž ***O(N+M)***, jelikož maximální velikost výsledného pole ***D_ST*** bude ***O(N)*** a do pole ***QUEST*** ukládáme hodnoty k otázkám, tedy ***O(M)***.

Ukázka implementovaného řešení:

Pro generování náhodných vstupů na ukázku se mi hodilo využít řešení falešné mince ze seriálu, díky kterému můžeme upravovat “pravděpodobnostní hustotu” znalostí studentů.

Zdrojový kód řešení je v C++:

```
#include <iostream>
#include <random>

using namespace std;

template<typename T>
T RandomProbability(T min, T max) {
```

```
    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<> dis(min, max);
    return dis(gen);
}

int GeneralCoinFlip(double p_1) {
    auto flip = RandomProbability<double>(0.0, 1.0);
    if (flip <= p_1) {
        return 1;
    }
    return 0;
}

vector<vector<int>> RandomInputGenerator(int N, int M) {
    vector<vector<int>> result (N, vector<int>(M, -1));
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            result[i][j] = GeneralCoinFlip(0.8);
        }
    }
    return result;
}

vector<int> BestStudentOrder(vector<vector<int>>& students) {
    vector<int> dumb_st;
    vector<int> smart_st;
    vector<int> questions(students[0].size(), 0);

    for (int i = 0; i < students.size(); i++) {
        for (int j = 0; j < students[0].size(); j++) {
            if (students[i][j] == 0) {
                questions[j]++;
            }
        }
    }

    int biggest_lack = -1, index = 0;
    for (int i = 0; i < students[0].size(); i++) {
        if (questions[i] > biggest_lack) {
            biggest_lack = questions[i];
            index = i;
        }
    }

    for (int i = 0; i < students.size(); i++) {
        if (students[i][index] == 0) {
            dumb_st.push_back(i);
```

```
        }
        else {
            smart_st.push_back(i);
        }
    }

    dumb_st.insert(dumb_st.end(), smart_st.begin(),
smart_st.end());
    return dumb_st;
}

int main() {

    vector<vector<int>> inputs = RandomInputGenerator(10, 10);

    cout << "Studenti a otázky: " << endl;
    for (int i = 0; i < inputs.size(); i++) {
        cout << i << "--> ";
        for (int j = 0; j < inputs[i].size(); j++) {
            cout << inputs[i][j] << " ";
        }
        cout << endl;
    }

    vector<int> order = BestStudentOrder(inputs);

    cout << "Výstup: " << endl;
    for (int i = 0; i < order.size(); i++) {
        cout << order[i] << "--> ";
        for (int j = 0; j < inputs[order[i]].size(); j++) {
            cout << inputs[order[i]][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```